LangChain Vs. LangGraph

When we want to implement a very complex, non-linear workflow then LangChain cannot alone help us do everything. Then, LangGraph was built on top of LangChain that helps us in creating these complex, non-linear workflows / agents. Below we will read about major challenges where LangChain struggles on its own during a complex workflow and how LangGraph overcomes those challenges —

*Challenge 1: Control Flow Complexity (Non-Linearity)

LangChain ⇒ It mostly supports linear executions (Chains). To implement loops (e.g. retrying something in a loop), conditional branching, or jumps, we must write custom python code outside the framework. this is all "Glue code" and high amounts of glue code make the application harder to maintain.
LangChain does have a feature called Runnable Branch and custom routing using LCEL (LangChain Expression Language) that allows for basic If/Else conditional routing. However, LCEL is completely acyclic - it cannot handle loops (cycles). LangGraph's distinct advantage is specifically handling cyclic graphs (loops).

LangGraph ⇒ It represents the workflow as a mathematical graph. Tasks are Nodes (standard python functions) and control flows are Edges. This natively supports loops and conditional branching (conditional_edges) without needing manual glue code.

*Challenge 2: State Handling

LangChain ⇒ It is inherently stateless. It does track conversational memory (chat history), but it does not natively track arbitrary application state. Tracking state requires manually maintaining global dictionaries and passing them through the chain.

Scanned with OKEN Scanner

LangGraph ⇒ It is inherently stateful. Here, we have to define a State object (a dictionary) when creating the graph. Every node reads from this state, performs its actions, and returns an updated state, which is automatically passed through to the next node.

- Defining State ⇒ In LangGraph, the "State" dictionary is strictly typed. In Python, we typically implement this using a TypedDict or a Pydantic BaseModel. Every node must return a dictionary that updates keys in this state.

Reducers ⇒ While updating State, LangGraph uses "reducers". By default, returning a new value overwrites the old value in the state. However, for other things like a list of chat messages, we can define a reducer (eg. Annotated [list, operator.add] which appends new messages to the existing list rather than overwriting it.

* Challenge 3: Event-driven Execution

LangChain ⇒ It executes sequentially. Once a chain starts, it runs to completion. It cannot natively pause for external triggers

LangGraph ⇒ It supports event-driven execution using **Checkpointers**. We can save the current state, pause the workflow, and seamlessly resume ~~for~~ from the exact paused node when an external trigger occurs.

* Fault Tolerance (Challenge 4)

LangChain ⇒ LangChain offers a .with_fallbacks ( ) method for specific runnables (e.g. if OpenAI fails, fall back to Anthropic). However, if a long-running multi-step chain fails at some intermediate stage, we have to restart the entire chain from begining. LangGraph ⇒ It provides native retry logic for small errors (like API downtime). For major system crashes, it offers recovery capabilities. Because the state is continuously saved via a persistence layer, the graph can resume execution from the exact node where the crash occured.

Scanned with OKEN Scanner

* Challenge 5: Human-in-the-loop

LangChain ⇒ We can ask for a quick human input during a chain, but pausing a script indefinitely (e.g., waiting 24 hours for some user action) consumes compute resources and risks crashing.

LangGraph ⇒ It treats human-in-the-loop as a core feature. It saves the graph state via checkpointers, safely pauses execution indefinitely without resource drain, and resumes once human input is received.

* Challenge 6: Nested Workflows (Subgraphs)

LangGraph feature ⇒ LangGraph allows us to place an entire graph inside a graph (inside a single node of larger graph). This is called a Subgraph. It is highly useful for Multi-Agent Systems and for reusability. LangChain does not support this level of encapsulation.

* Challenge 7: Observability

LangChain ⇒ While integrated with LangSmith for monitoring LLM calls, LangSmith cannot see or track the custom Python "glue code" used for loops and conditions. This results in partial observability.

LangGraph ⇒ Because there is zero glue code and the state is explicitly passed node-by-node, LangSmith can log a complete, chronological timeline of the entire execution, making debugging highly efficient.

When to use What?

1) LangGraph does not replace LangChain. It is built on top of it.
2) We still have to use LangChain components (Models, Prompts, Retrievers, Output Parsers) to do the actual LLM work inside LangGraph.
3) Use LangChain for straightforward, linear, sequence-based applications.
4) Use LangGraph when we need non-linear control flows, loops, statefulness, human-in-the-loop approvals, and multi-agent coordination.

Scanned with OKEN Scanner